Computers & Security 168 (2026) 104955

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Towards formal validation and performance evaluation of TLS 1.3 using Intelligent Transport System certificates

Sihem Baccari, Mohamed Hadded *

Abu Dhabi University, Abu Dhabi, United Arab Emirates

ARTICLE INFO

Keywords:
TLS 1.3
Intelligent Transport Systems
V2X communication
Formal verification
Security protocols
ITS certificates
ProVerif
Transport Layer Security
Public Key Infrastructure
Secure communication

ABSTRACT

Transport Layer Security (TLS) 1.3 introduces significant improvements in both performance and security compared to previous versions. However, its integration with domain-specific certificate schemes, such as those used in Intelligent Transport Systems (ITS), raises new challenges in protocol validation and trust establishment. This paper investigates the formal validation of TLS 1.3 in the context of vehicular communication environments that rely on ITS certificates. We analyze the compatibility between TLS 1.3's handshake and authentication mechanisms and the structure and policies of ITS certificate formats, particularly in Vehicle-to-Everything (V2X) scenarios. Using formal methods, we identify potential interoperability issues and propose a validation framework that incorporates ITS-specific constraints into the TLS trust model. To support and extend the formal analysis, we conduct an experimental performance evaluation comparing ITS certificate validation with standard X.509 ECC certificates. The experimental results show that ITS certificates achieve comparable validation latency while providing a 68% reduction in certificate size. Our approach aims to ensure secure and standards-compliant deployment of TLS 1.3 in ITS environments, contributing to enhanced security and trust in emerging intelligent transportation infrastructures.

1. Introduction

Today's world is experiencing widespread digital interconnection, making the security and confidentiality of exchanged data a major challenge. The TLS cryptographic protocol plays a vital role in securing networks and protecting them against potential threats. It ensures the reliability of the communicating parties, allows encrypted data exchange, and prevents malicious modification or interception of flows using advanced cryptographic mechanisms (Krawczyk and Wee, 2016; Li et al., 2016). The beginnings of TLS date back to the 1990s, when the growth of the internet required the implementation of robust protocols to secure the information exchange. It evolved from the Secure Sockets Layer (SSL) protocol developed by Netscape. Thus, after the release of SSL 2.0, several security vulnerabilities were discovered and explored due to plaintext negotiation of cryptographic algorithms and weak integrity mechanisms (Satapathy et al., 2016; Sirohi et al., 2016; Eldewahi et al., 2015). These weaknesses have quickly led to the release of a new stable and secure version, SSL 3.0 in 1996, which served as the basis for TLS. In 1999, the Internet Engineering Task Force (IETF) released TLS 1.0 (RFC 2246 (Dierks and Allen, 1999)) with an architecture similar to SSL 3.0, which uses more robust algorithms. TLS continued its evolution with TLS 1.1 (RFC 4346 (Dierks and Rescorla, 2006)) in 2006 and TLS 1.2 (RFC 5246 (Dierks and Rescorla, 2008)) in 2008, which introduced the use of more secure hashing algorithms such as SHA-256. TLS 1.3 (RFC 8446 (Rescorla, 2018)) is the latest version of the TLS protocol released in 2018. This version brought significant improvements in both security and performance, including the removal of obsolete cryptographic mechanisms (e.g. RSA key exchange, SHA-1, CBC) and reduced latency due to the zero-Round Trip Time (0-RTT) mechanism (Lee et al., 2021; Abdelhafez et al., 2023). Currently, TLS with its version 1.3 is the most recommended protocol in various applications and services, and its adoption is crucial to protect privacy and ensure digital trust (Zhou et al., 2024). These reasons make TLS today a key candidate for providing end-to-end security within ITS systems.

V2X technologies rely essentially on the massive exchange of real-time data between different ITS network entities (Boulila et al., 2018). These critical and time-sensitive data require permanent protection against privacy attacks and service availability threats (Hadded et al., 2019), making the TLS protocol an essential solution to secure vehicular communications. Basically, ITS systems use lightweight digital certificates defined by the IEEE 1609.2 (IEEE, 2016) and ETSI TS 103 097 (ETSI, 2017) standards. On the other hand, TLS relies on the use of classic certificates of type X.509 and does not natively

* Corresponding author.
E-mail addresses: sihembaccari51@gmail.com (S. Baccari), mohamed.elhadad@adu.ac.ae (M. Hadded).

https://doi.org/10.1016/j.cose.2026.104955
Received 18 October 2025; Received in revised form 7 April 2026; Accepted 4 May 2026
Available online 15 May 2026

support ITS certificates. Thus, to meet the strict requirements of ITS communications, RFC 8902 (Msahli et al., 2020) defined a new usage of ITS certificates in the context of mutual authentication via TLS. This specification defines how to introduce 1609.2 certificates into the TLS handshake process and also addresses the interoperability between TLS and ITS Public Key Infrastructure (PKI) system. This integration is further endorsed by ISO 21177:2024 (ISO, 2024), which specifies the establishment of secure sessions for trusted ITS devices using TLS 1.3 and ITS certificates via RFC 8902.

The growing complexity of vehicular communications interactions raises concerns about the robustness of TLS using ITS certificates against implementation errors and compatibility issues. For this, it is essential to study the implications of integrating ITS certificates instead of X509 certificates in TLS 1.3 handshake and the potential security vulnerabilities. Hence, verification using formal methods allows validating the effectiveness of the integration of ITS certificates in TLS, to prove their resistance to replay attacks due to the particularities of ITS certificates, and to identify possible security issues that they may face.

1.1. Related work

The formal analysis of TLS has received significant attention from the research community. Thus, several works have investigated the formal verification of TLS 1.3 using automated tools such as the Tamarin prover, providing strong guarantees on its core security properties under various threat models, with notable contributions in Cremers et al. (2017), Bhargavan et al. (2017), Cremers et al. (2016). These studies, however, focus on TLS in its standard deployment context and assume the use of X.509 certificates within conventional PKI hierarchies. In parallel, a growing body of research has addressed security challenges in V2X communications, particularly focusing on PKI architectures, credential management systems, and message authentication mechanisms defined in IEEE 1609.2 and ETSI standards, as discussed in Brecht et al. (2018), Yoshizawa et al. (2023), Bussa et al. (2025).

To date, despite these advances, the integration of TLS 1.3 with ITS certificate infrastructures as specified in RFC 8902, has not been studied from a formal verification perspective. In particular, existing works either focus on the core TLS protocol or on V2X security mechanisms in isolation, without addressing their combined use.

In this context, to the best of our knowledge, this paper presents the first formal verification study of the ITS-TLS protocol, allowing to validate the expected security guarantees, such as mutual authentication and the proper compliance with application permissions associated with ITS certificates, and proposes some recommendations for its practical implementations. Our study complements existing verification efforts by addressing the specific challenges arising from the convergence of two distinct security ecosystems.

1.2. Contributions

In this work, we provide the first effort at formal analysis of ITS-TLS integration and make the following key contributions:

1. We developed the first formal model of ITS-TLS, for the TLS 1.3 handshake using ITS certificates, capturing the complex protocol semantics and permission checks according to the RFC 8902 and the IEEE 1609.2 specification.
2. Using ProVerif, we formally verify critical security properties such as session key confidentiality, mutual authentication via ITS certificates, certificate validity and authorization via PSID and SSP,[1] and crucially the correct enforcement of context binding. Our analysis shows that these guarantees are met when properly implemented.

[1] The Provider Service Identifier (PSID) specifies the class of service or application for which a certificate is authorized. The Service Specific Permissions (SSP) define the permissions and constraints associated with that PSID (IEEE, 2016).

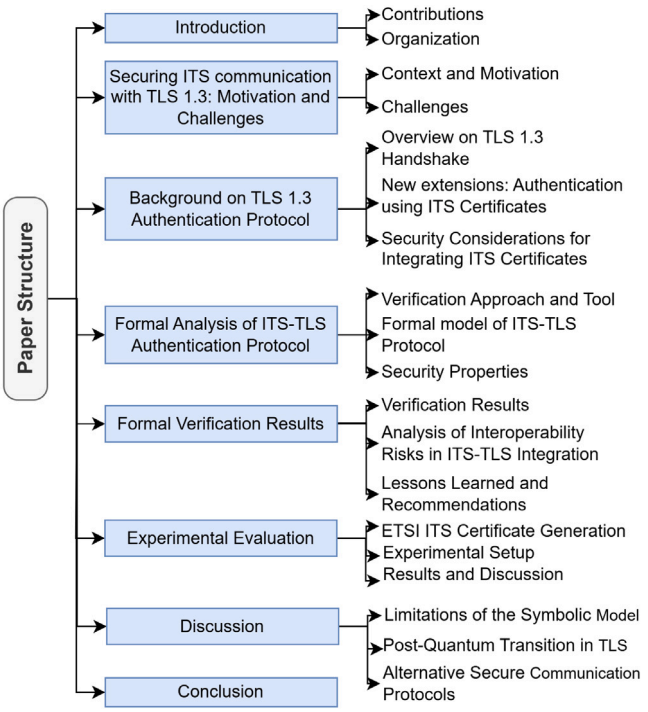

Fig. 1. Structure and organization of the paper.

3. We demonstrate inherent sources of ITS context-specific vulnerabilities arising from the complexity of the ITS certificate validation process, mainly new vectors for denial-of-service attacks. We discuss additional risks of semantic confusion attacks through improper HeaderInfo validation.
4. We conduct an experimental performance evaluation, comparing ITS certificate validation overhead against standard X.509 ECC implementations.
5. We outline essential future directions and propose recommendations to strengthen ITS-TLS security, emphasizing the need to assess mobility impacts on handshake latency while highlighting the new challenges posed by the upcoming post-quantum transition.

1.3. Organization

The remainder of the paper is organized as follows: Section 2 presents the motivation and challenges of securing ITS communications with TLS 1.3. Section 3 provides background on the TLS 1.3 handshake and the proposed extensions for ITS certificate integration. Section 4 details the formal analysis of ITS-TLS. Section 5 reports the formal verification results. Section 6 presents an experimental performance evaluation. Section 7 discusses broader perspectives, including symbolic model limitations, post-quantum transition, and alternative protocols. Finally, Section 8 concludes the paper. An overview of the organization and the full content flow is illustrated in Fig. 1.

2. Securing ITS communication with TLS 1.3: Motivation and challenges

2.1. Context and motivation

With the evolution of ITS, vehicle-to-vehicle (V2V), vehicle-to-infrastructure (V2I), and more broadly V2X communications, connectivity is becoming a critical component for traffic efficiency and the emergence of innovative services. Thus, the core of this ecosystem

relies on the continuous exchange of critical messages, such as Cooperative Awareness Messages (CAMs) and Decentralized Environmental Notification Messages (DENMs), between different V2X entities. Hence, the integrity and confidentiality of these messages are fundamental prerequisites for ensuring public safety (Ali et al., 2024). For instance, a malicious entity that spoofs CAMs could create virtual ghost vehicles, leading to catastrophic accidents, while eavesdropping on V2X data streams poses serious risks to drivers' privacy (Sedar et al., 2023).

Basically, to secure V2X communications, the ITS community has relied on the IEEE 1609.2 standard. This standard defines a security infrastructure based on a PKI system and specifies the use of dedicated ITS certificates, which are designed to be processed efficiently to meet the low-latency requirements of safety-critical applications. These certificates are typically attached directly to each V2X message for direct verification (Lekidis and Morais, 2024; Farran and Khoury, 2023). However, the ITS landscape is evolving increasingly rapidly beyond isolated and pure V2X interactions, creating a compelling need for additional security mechanisms.

Currently, the emergence of hybrid communication architectures, where different communication technologies, such as ITS-G5/DSRC (802.11p, 2010), C-V2X (LTE-V2X, 5G-NR V2X) (3GPP, 2025b,a), and traditional cellular networks (4G/5G), coexist creating heterogeneous environments. Vehicles will need to access more remote services (e.g. software updates, high-definition map ingestion, etc.) and interact with cloud platforms (Baccari et al., 2025). These client–server interactions fall outside the scope of the pure broadcast-oriented IEEE 1609.2 security model. For this, the rise of service-oriented applications, which go beyond safety applications, creates a new class of services that require reliable and confidential data channels. These applications include infotainment services, real-time traffic analytics from infrastructure sensors, secure payment, and over-the-air (OTA) updates (Clancy et al., 2024). These services have security requirements that align perfectly with the capabilities of the TLS protocol.

To this end, RFC 8902 becomes critically important. As it helps ensure complementarity between ITS security model and the TLS 1.3 protocol. It discusses how an ITS station, e.g. a vehicle or a Road Side Unit (RSU), could authenticate itself to a TLS server (e.g. a service provider) using its native and standardized ITS certificate instead of a traditional X.509 certificate. This integration is vital for several reasons, as it essentially paves the way for a new generation of secure ITS services and ensures strong end-to-end security in an increasingly heterogeneous ecosystem. Therefore, the motivation for integrating TLS 1.3 with ITS certificates is not to replace IEEE 1609.2 but to complement it, creating a holistic security framework that covers both real-time safety-critical and future non-safety services.

2.2. Challenges

Although the concept and rules defined in RFC 8902 are presented in a comprehensive manner, the practical implementation and integration of TLS 1.3 into the ITS ecosystem raises specific technical challenges that must be thoroughly addressed, mainly:

- **Latency and Resources Constraints:** Establishing a TLS 1.3 handshake that involves mutual authentication with ITS certificates may introduce additional computational overhead. Thus, TLS 1.3 relies on computationally intensive cryptographic primitives, which can affect the system responsiveness and energy efficiency of a resource-constrained ITS station, if not optimized through hardware acceleration or tailored implementations. Furthermore, the certificate chain validation process could introduce network latency, since ITS certificates are encoded using a complex format compared to a standard TLS certificate. However, safety messages require end-to-end delivery within a few milliseconds (Hadded et al., 2017). Although TLS 1.3 reduces handshake latency compared to previous versions, the time to establish a secure TLS session may still be too high for highly time-sensitive ITS applications, where the acceptable latency must strictly not exceed 100 ms, as recommended by leading organizations such as the 5G Automotive Association (5GAA) in its C-V2X service level specifications ((5GAA), 2023), the ETSI in its ITS standards related to CAM/BSM (ETSI, 2019), and the 3rd Generation Partnership Project (3GPP) in its specifications for Ultra-Reliable Low Latency Communication (URLLC) requirements (3GPP, 2020). Furthermore, several industrial and institutional stakeholders target even stricter objectives, in the range of 20 to 50 ms, to meet critical use cases such as collision avoidance or cooperative emergency braking ((5GAA), 2018; Alliance, 2018).
- **Short-Validity Certificates and Mobility Constraints:** ITS certificates are designed with very short validity periods to limit the impact of potential compromises. While this reinforces the security, it complicates the authentication process in TLS, which typically relies on longer-lived certificates. Additionally, TLS was originally designed for stable client–server connections for a longer period. In ITS environments, vehicles and roadside units frequently change communication partners, often within seconds (Hadded et al., 2020). Therefore, it is important to ensure that the TLS 1.3 handshake mechanisms can efficiently support the establishment and termination of rapid sessions, while maintaining its strong security guarantees.
- **Certificate Validation and Scalability:** The validation process for an ITS certificate is more complex than for a conventional X509 certificate. These certificates require additional checks for specific permissions for different applications in ITS. Checks also include geographic validity regions and certificate revocation status, which is often handled using different structures rather than the Online Certificate Status Protocol (OCSP) common in web PKI (Chen, 2023). The TLS server must integrate a dedicated ITS certificate validation module that understands these unique semantics, which is not part of standard TLS libraries. Moreover, TLS is designed primarily for persistent client–server interactions and is not inherently optimized for such massive numbers of ephemeral connections. This creates challenges in maintaining both security and scalability.

3. Background on TLS 1.3 authentication protocol

In this section, we present an overview of the TLS 1.3 handshake protocol, highlighting the adaptations necessary for the integration of certificates compliant with the ITS context, as well as the specific security requirements that this integration entails.

3.1. Overview on TLS 1.3 handshake

Compared to previous versions, TLS 1.3 introduces a simplified and more secure mechanism to establish a secure connection between two parties, client and server, while minimizing the number of round trips (Cremers et al., 2017). The TLS 1.3 handshake, representing the initial phase of the protocol, allows essentially for the negotiation of cryptographic parameters between the two parties, the exchange of ephemeral keys to generate session secrets, and the authentication of at least one party, typically the server, using a digital certificate. Indeed, the handshake process relies on a structured exchange of two main phases: an initial unencrypted phase in which session keys are negotiated via *ClientHello* and *ServerHello* messages, followed by an encrypted phase where authentication and key derivation messages are exchanged (Dowling et al., 2021).

The *ClientHello* message, which is the handshake trigger, consists of several fields, including mainly a `cipher_suites` field indicating the cryptographic algorithms supported by the client, and the extensible `extensions` field which encompasses in particular: `supported_versions` (expressing that TLS 1.3 is supported),

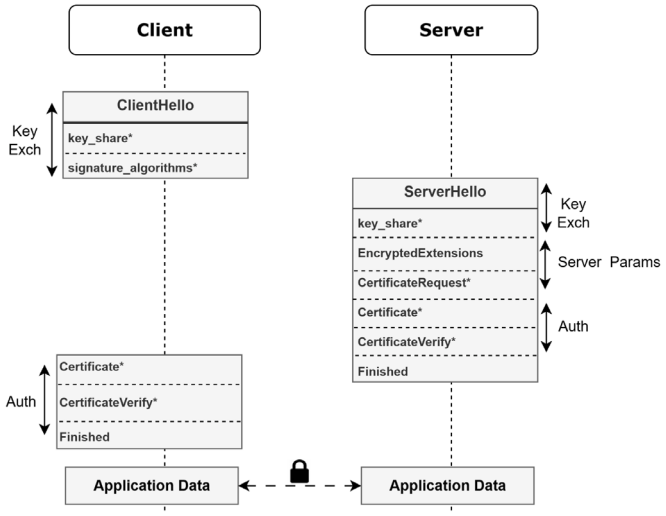

Fig. 2. Message flow for full TLS 1.3 handshake.

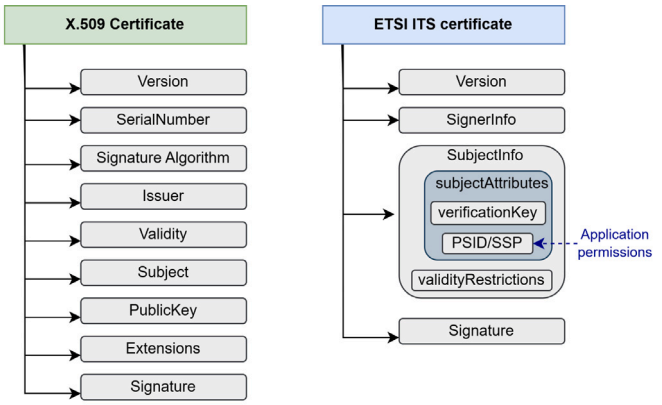

Fig. 3. Structural comparison between classical X.509 certificates and ETSI TS 103 097 certificates.

`key_share` containing public key for negotiating of the shared secret, `signature_algorithms` (accepted signature algorithms), `supported_groups`, and possibly `server_name (SNI)` to identify the target server. In response, the *ServerHello* message references the *ClientHello* structure with a specified selected value. It includes in particular the `cipher_suite` field which contains a suite chosen from those proposed by the client, and the extensions field which mainly contains the `key_share` corresponding to the chosen group, as well as `supported_versions` to confirm the use of TLS 1.3 (Rescorla, 2018; Zhou et al., 2024). Together, these fields allow both parties to derive a secure shared secret, thus initiating the encrypted phase of the handshake. Hence, following this exchange, both parties have a shared secret via Elliptic Curve Diffie–Hellman Ephemeral (ECDHE) from which the handshake keys are derived using HMAC-based Key Derivation Function (HKDF) (Sardar et al., 2024; Krawczyk and Eronen, 2010).

As shown in Fig. 2, the full standard handshake flow for TLS 1.3 is summarized as follows:

(1) The client starts by sending a *ClientHello* with its ECDHE public key and cryptographic preferences.

(2) The server responds with a *ServerHello*, selects the parameters, and sends its ECDHE key.

(3) A shared secret is calculated via ECDHE, and then used in an HKDF scheme to derive the symmetric keys for the handshake phase.

(4) The server then sends a set of encrypted messages using the handshake keys:

- *EncryptedExtensions:* additional negotiated information.
- *Certificate:* its certificate chain, typically in X.509 format.
- *CertificateVerify:* a signature on the handshake transcript.
- *Finished:* proof of authenticity using a Message Authentication Code (MAC).

(5) The client verifies all of these messages, responds with Certificate and CertificateVerify if requested by the server, and then sends its own Finished.

3.2. New extensions: Authentication using ITS certificates

3.2.1. Overview of ITS certificates and integration objectives

The adoption of ITS-specific certificates, over traditional X.509 certificates, is driven by the unique security, privacy, and performance requirements of vehicular communication. These networks operate under several major constraints of high mobility and strong privacy challenges, which X.509 certificates were not designed to meet. Thus, the traditional use of X.509 certificates as standardized in TLS for ITS environments presents multiple issues that can hinder their efficient use. As shown in Table 1, these certificates suffer mainly from the large size of the certificate, where no native pseudonymity mechanism is supported to preserve the real identity of the holder. Indeed, X.509 certificates are designed for long-term identity binding (Berbecaru and Lioy, 2023), which makes them unsuitable for dynamic vehicular environments where privacy preservation is a prerequisite. Furthermore, reliance on a hierarchical PKI system and centralized revocation mechanisms (Adja et al., 2021) increases the complexity of certificate management in real-time and latency-sensitive environments.

On the other hand, ITS certificates are designed mainly to support low-latency and bandwidth-constrained communications, making them more suitable for V2X networks. They feature a more compact structure and a lighter size compared to X.509 certificates. In addition, ITS certificates frequently adopt a pseudonymized scheme, where the identity of the holder cannot be directly revealed and the certificate validity is intentionally short (Chen et al., 2023, 2024). This reduces the need for complex revocation mechanisms and optimizes verification latency. For this, the goal of integrating ITS certificates into TLS 1.3 is to enable privacy-preserving authentication and mutual trust establishment between V2X network entities. This allows vehicular systems to benefit from the proven security guarantees of TLS while adhering to the lightweight and decentralized identity model enforced in ITS networks. Overall, the ITS schema prioritizes compactness and parsing determinism at the cost of flexibility, whereas X.509 favors extensibility and generic applicability. Fig. 3 illustrates this contrast between the extension-centric X.509 model and the compact permission-centric ITS certificate format.

3.2.2. TLS handshake extensions for ITS certificate support

To support the integration of IEEE 1609.2 certificates within the TLS 1.3 handshake, RFC 8902 introduces two critical handshake extensions as defined in RFC 7250 (Wouters et al., 2014). These extensions, `client_certificate_type` and `server_certificate_type`, allow peers to explicitly negotiate the type of certificate encoding supported by each endpoint, and thus enable the server authentication and optionally the client using ITS certificates instead of traditional X.509. During the handshake, the client includes both extensions in the *ClientHello* message, where each contains an ordered list of preferred supported *CertificateType* values, typically between X.509 and 1609Dot2, for the server and client certificates, respectively. For example, the client may require in its *ClientHello* message as follows:

client_certificate_type=(1609Dot2),

Table 1
Comparison between ITS and X.509 certificates.

Aspect	ITS Certificate	X.509 Certificate
Standard	IEEE 1609.2/ETSI TS 103 097	RFC 5280
Size	< 400 bytes	> 1000 bytes
Validity Period	Short (minutes/hours)	Long (months/years)
Privacy	Supports pseudonymity	Fully identifiable subject
Permissions	Explicit service permissions in certificate	No native permissions
Main Fields	CertType (implicit/explicit), Issuer's Identifier, Geographic Region, AppPermissions (PSID/SSP), Public Key, Validity Period	Issuer Name, Validity (not before/not after), Subject Name, Public Key Info, Key Usage
Certificate Usage Control	Controlled via PSID/SSP, region and time restrictions	Optional via KeyUsage, ExtendedKeyUsage extensions
PKI Model	Decentralized/Multi-role	Centralized/Hierarchical
Revocation Mechanism	Lightweight (short revocation lists (CRLSeries) and limited certificate validity)	Classic revocation using CRL/OCSP

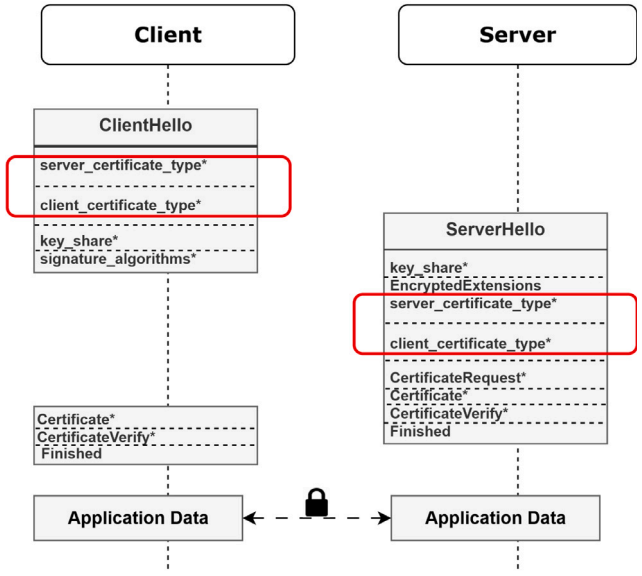

Fig. 4. Message flow with new certificate type extensions for full TLS 1.3 handshake.

server_certificate_type=(1609Dot2, X509, RawPublicKey)

The server processes these extensions and selects one certificate type from each list. The selected values are then returned to the client in the `EncryptedExtensions` message, completing the certificate format negotiation phase.

As shown in Fig. 4, once the client and server have agreed on the 1609.2 types, the `Certificate` and `CertificateVerify` handshake messages carry Ieee1609Dot2Data encoded according to the IEEE 1609.2 standard using Octet Encoding Rules (OER), rather than the standard Distinguished Encoding Rules (DER) used for X.509 certificates. Furthermore, the included extensions mechanism enables backward compatibility, when no agreement on 1609.2 is reached, the handshake can gracefully fall back to traditional X.509 certificates, allowing coexistence in mixed environments. Thus, in practice, two configurations are possible when using ITS certificates with TLS. First, both the client and the server can use 1609.2 certificates by including the `client_certificate_type` and `server_certificate_type` extensions with the value Ieee1609Dot2, allowing mutual authentication using ITS credentials. Alternatively, the client can support X.509 certificates by listing X.509 or RawPublicKey in the `server_certificate_type` extension. In this case, the server authenticates with an X.509 certificate, while the client can still use an ITS certificate if the server accepts it. This allows for flexible use of different certificate formats depending on system requirements.

3.3. Security considerations for integrating ITS certificates

Although the included extensions provide a lightweight and flexible adaptation of TLS for V2X systems, they introduce specific security considerations in the authentication phase. Thus, as shown in Fig. 5, these implications mainly affect `CertificateVerify` message, which requires including a set of specific data in TLS certificates. We summarize these changes to be considered as follows:

- **Certificate Verification Considerations:** when peers agreed on the `certificate_type` is 1609Dot2, the certificate verification process differs significantly from the traditional X.509 validation. In this case, the `CertificateVerify` message must include a structure of Ieee1609Dot2Data, which encapsulates the digital signature along with a `headerInfo` field that contains a critical contextual information. As indicated in Fig. 6, this field includes metadata such as `psid` which gives insight into the application activity for which the certificate is authorized, `generationTime` at which the data structure was created, and `pduFunctionalType`. To comply with the TLS standard, the `pduFunctionalType` must be set to *tlsHandshake*, ensuring that the signature is strictly bound to the handshake context and cannot be misused in application layer communications.
- **Permissions and PSID Validation:** Contrary to X.509 certificates, ITS certificates explicitly encode the permissions granted to the entity through the pair of PSID and SSP in the `appPermissions` field. During handshake processing, the certificate must contain a valid `appPermissions` field authorizing the use of the declared PSID to sign TLS handshake messages. If such an authorization is missing or invalid, the certificate must be rejected.
- **Certificate Freshness Verification:** As ITS certificates are designed with a short validity period of a few hours, each peer in the TLS session that accepts 1609Dot2 certificates needs to check the expiration time in the received certificate and terminates the session when its validity has expired. Hence, avoid relying on traditional revocation mechanisms using CRLs or OCSP (Santesson et al., 2013), and instead ensure freshness through the expiration time. However, in some cases, the complete certificate chain may not be transmitted during the TLS handshake. To support this, RFC 8902 recommends the use of secure online repositories or trusted local caches, as defined in ETSI TS 102 941 (ETSI, 2018), to obtain the necessary ITS certificates and CA chains in a secure and authenticated manner.
- **Cryptographic Strength and Interoperability:** In hybrid handshake session where both 1609Dot2 and X.509 certificates are used for authentication, RFC 8902 recommends that all cryptographic primitives, regardless of the certificate type, should provide equivalent or stronger security guarantees to ensure consistent protection throughout the entire TLS session. Since ITS certificates rely essentially on Elliptic Curve Cryptography (ECC) to ensure a minimum of 128-bit security. For this, to avoid a breach in the security level, X.509 certificates must use algorithms as strong as those used for ITS certificates.

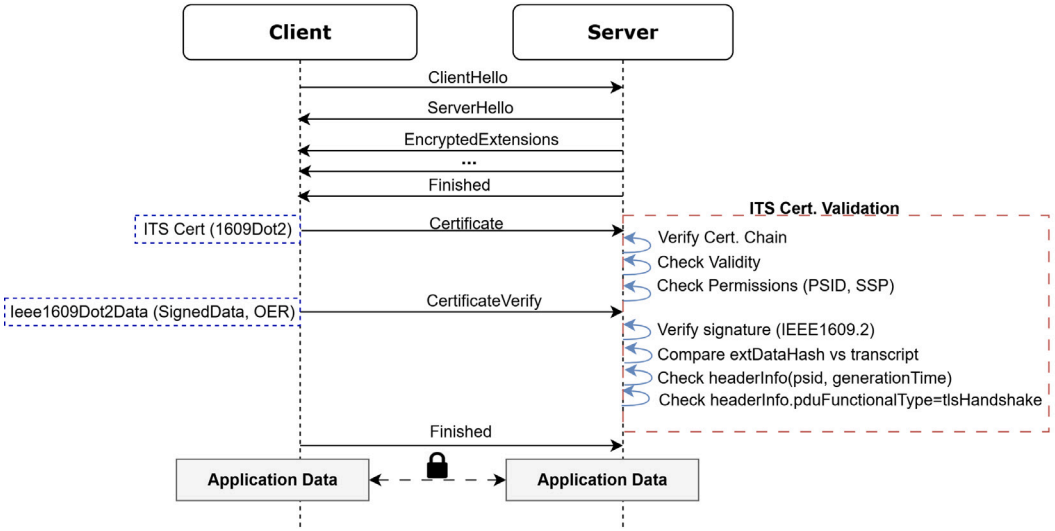

Fig. 5. TLS 1.3 CertificateVerify validation flow with IEEE 1609.2 ITS certificate structures.

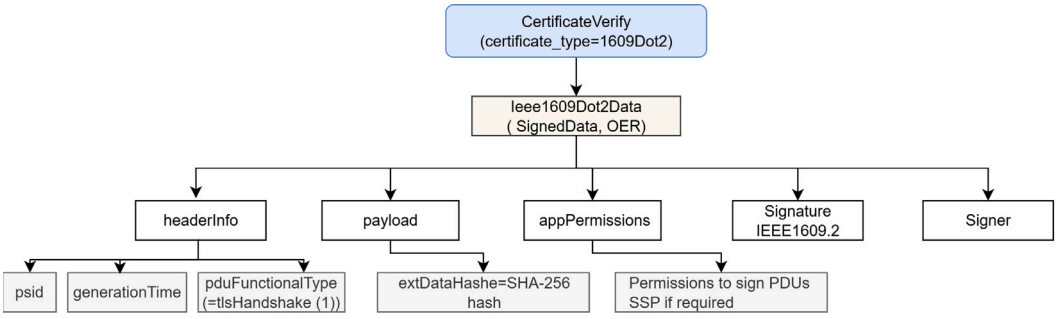

Fig. 6. Structure of TLS 1.3 CertificateVerify message using IEEE 1609.2 ITS certificates.

4. Formal analysis of ITS-TLS authentication protocol

In this section, we present a symbolic formalization of the ITS-TLS handshake protocol, based on the extensions introduced in RFC 8902 for the use of IEEE 1609.2 certificates. The model is written in the applied pi-calculus and targets verification using ProVerif tool. The model captures the interaction between client and server during a full TLS 1.3 handshake, including certificate type negotiation and mutual authentication using ITS certificates. We also present the formalization of the intended security properties.

4.1. Verification approach and tool

To formally verify the security properties of the TLS handshake protocol extended with IEEE 1609.2 certificates, we adopt a formal verification methodology based on symbolic protocol analysis. This approach allows exploring possible executions of the protocol, under the Dolev–Yao model (Dolev and Yao, 1983), which provides a symbolic abstraction of how an attacker can interact with messages on a network. For this purpose, we use ProVerif (Blanchet et al., 2022, 2018; Blanchet, 2001), a widely used automated tool for the formal analysis of cryptographic protocols (Lafourcade and Puys, 2016). ProVerif models protocols using a process calculus derived from the applied pi-calculus (Abadi et al., 2018; Blanchet, 2016), and supports the specification of both the protocol logic and the attacker capabilities. It allows verification security properties such as secrecy of session keys, authentication of endpoints, and correspondence assertions through symbolic reasoning.

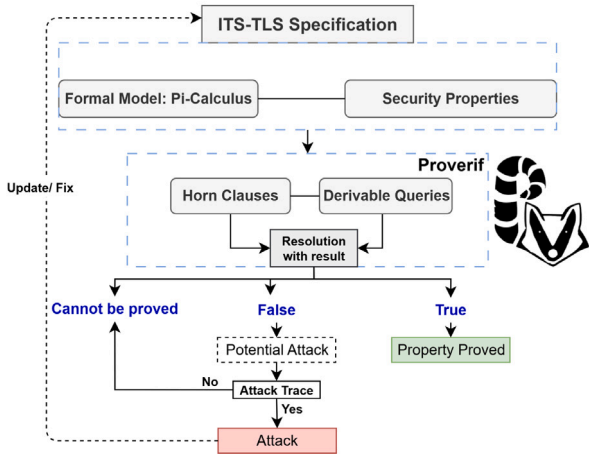

Fig. 7. Formal verification process of the ITS-TLS handshake using ProVerif.

In the context of our work, we employ ProVerif to formally model the ITS-TLS handshake protocol as described in RFC 8902, including the negotiation of certificate types via `client_certificate_type` and `server_certificate_type` extensions, and the updated structure of the `Certificate` and `CertificateVerify` messages. The model also captures the use of permission-based fields

using psid/ssp, and the contextual binding of signatures via pduFunctionalType, which are essential to authenticate using ITS certificates. As depicted in Fig. 7, this formal approach can help identify potential vulnerabilities and assess whether the intended security guarantees discussed above are preserved in the presence of a malicious attacker (Zhang et al., 2020).

4.2. Formal model of ITS-TLS protocol

4.2.1. Modeling assumptions and abstractions

The goal of this formal model is to capture the essential behavior of the TLS 1.3 handshake as specified in RFC 8902, in order to verify its security properties using symbolic analysis. Features such as session resumption, 0-RTT, and renegotiation are excluded, as they fall outside the scope of certificate-based authentication.

Cryptographic Assumptions: Given the complexity of real-world protocol implementations, cryptographic primitives are assumed to be perfect where hash functions are ideal, digital signatures cannot be forged, and encryption cannot be broken. We adopt the Dolev–Yao attacker model, in which the adversary has complete control over the communication network and can intercept, replay, as well as inject messages, but cannot break cryptographic assumptions only by possessing the correct key (Zhang et al., 2020).

Cryptographic Primitives: To model the cryptographic operations and certificate logic of the ITS-TLS handshake, we abstract cryptographic primitives using symbolic functions in the applied pi-calculus. The model includes standard public-key primitives, key derivation, and asymmetric encryption, declared in Proverif as follows:

```
fun pk(privateKey): publicKey.
fun aenc(bitstring, publicKey): bitstring.
fun sign(bitstring, privateKey): bitstring.
fun hash(bitstring): bitstring.
fun hkdf(bitstring, bitstring): bitstring.
```

These primitives are used to model expected cryptographic behavior, including encryption/decryption as well as signing and signature verification, as illustrated by the following ProVerif reductions:

```
reduc forall m: bitstring, k: privateKey; adec(aenc(
    m, pk(k)), k) = m.
reduc forall m: bitstring, k: privateKey; checksign(
    sign(m, k), pk(k)) = m.
```

Communication Channel: All protocol messages between the client (typically a vehicle) and the server (e.g., RSU, traffic management center, etc.) are transmitted over a public channel, which is accessible to the attacker. This channel models insecure network communication that is consistent with the standard symbolic threat model, as follows:

```
free main_channel : channel.
```

Certificate Type Negotiation: The two extensions, *client_certificate_type* and *server_certificate_type*, are modeled as constants used in *ClientHello* and *ServerHello* messages to negotiate the use of ITS certificates, as follows:

```
const server_cert_type_its : extension.
const client_cert_type_its : extension.
```

ITS Certificate Abstraction: ITS certificates are abstracted to contain mainly the essential fields for authentication and authorization, including: the identity of the certificate holder identity and its public key publicKey, two symbolic fields timestamp represent the start and end of the certificate validity period and model the behavior of a short-term certificate, a set of itsPermissions constructed from a couple of psid and ssp, and the name of the issuing certificate authority caName. This is modeled using ProVerif by employing the following functions:

```
fun makePermissions(psid, ssp): itsPermissions.
fun generateITScert(identity, publicKey, timestamp,
     timestamp, itsPermissions, caName):
    itsCertificate.
```

Time and Validity Modeling: To ensure consistent certificate verification, time is modeled as a symbolic value currentTime of type timestamp, and expiration is enforced via a comparison using the function isBeforeOrEqual which simulates a symbolic comparison defined by a simple reduction as follows:

```
reduc forall t: timestamp; isBeforeOrEqual(t, t) =
    true.
```

The certificate's expiration time is extracted using two functions, extractTimestamp and extractExpiration, which retrieve the certificate's start and expiration dates, respectively. Thence, validity is verified using two symbolic comparisons checkStart and checkEnd. Finally, verifyCASignature completes the certificate verification by ensuring that it was issued by the root ITS certification authority. This is symbolically designed in ProVerif as follows:

```
let certStart = extractTimestamp(Cert) in
let certEnd = extractExpiration(Cert) in
let checkStart=isBeforeOrEqual(certStart,
    currentTime) in
let checkEnd = isBeforeOrEqual(currentTime, certEnd
    ) in
let sigValid = verifyCASignature(Cert, rootCA_ITS)
    in
...
```

Permission Verification: PSID/SSP permission verification is reinforced by the checkAppPermission() function, which ensures that the PSID and the associated SSPs match the authorizations required to initiate a TLS handshake, defined in ProVerif as follows:

```
fun checkAppPermission(psid, pduFunctionalType, ssp
    ): bitstring.
let perms = extractPermsFromCert(Cert) in
let psid = extractPSID(perms) in
let ssp = extractSSP(perms) in
let checkperm = checkAppPermission(psid,
    tlsHandshake, ssp) in ...
```

Context Binding: To correctly bind the signature in the CertificateVerify message to the ITS-TLS handshake context, we follow RFC 8902's requirement to include the *pduFunctionalType* field. This is modeled as a symbolic constant tlsHandshake of type pduFunctionalType. Then, this constant is used in the computeITSVerify function, which simulates the generation of the ITS signature added to the *CertificateVerify* message. This explicitly anchors the signature in the context of the TLS protocol, as required by the ITS standard. This contextual binding invalidates any attempt to reuse the signature in another context. The signature is generated as follows:

```
const tlsHandshake : pduFunctionalType.
out(main_channel, certificateVerify(
    computeITSVerify( toTranscriptHash(...),
    PrivKey,PSID,tlsHandshake)));
```

On the receiving side, the verifier reconstructs the signed data by hashing the same structure combined with the expected PSID and the fixed tlsHandshake value. The received signature is then verified using the sender's public key, ensuring both authenticity and correct context binding.

4.2.2. Client process

The client process models the case of an ITS-enabled vehicle initiating a TLS 1.3 negotiation using an ITS certificate. It starts by generating a fresh key pair (private and public keys), which is then used to construct an ITS certificate, with its identity and permissions, and signed by a trusted CA, as follows:

```
let clientProcess =
new clientPrivKey: privateKey;
let clientPubKey = pk(clientPrivKey) in
new clientPSID: psid;
new clientSSP: ssp;
let clientPerms=makePermissions(clientPSID,
    clientSSP) in
let clientCert = generateITScert(clientId,
    clientPubKey, currentTime, certValidityPeriod,
    clientPerms, itsRootCA) in
```

After generating its certificate, the client initiates the handshake by sending a *ClientHello* message containing its nonce and extension preferences. In parallel, the `beginHandshake(clientId)` event is triggered to mark the start of the authentication protocol on the client side, as follows:

```
new clientNonce: nonce;
event beginHandshake(clientId);
out(main_channel, clientHello( clientNonce,
    server_cert_type_its ));
```

Then, the client receives sequentially the expected TLS messages from the server, namely ServerHello, EncryptedExtensions, Certificate, etc. This sequence is modeled in ProVerif using a series of entries on the main channel (`in(main_channel, . . . )`) representing each message in the TLS 1.3 protocol, as follows:

```
in(main_channel, serverHelloMsg: tlsMessage);
...
in(main_channel, certReqMsg: tlsMessage);
in(main_channel, rawServerCertMsg: tlsMessage);
let certificateMsg(serverCert) = rawServerCertMsg
    in
...
in(main_channel, serverFinishedMsg: tlsMessage);
```

Upon receiving the server's certificate, the client performs certificate validation steps as detailed previously. If all conditions are met regarding validity and permissions, the client triggers the `certValidated(serverId)` event indicating that the server certificate is recognized as authentic and the `receivedCert(serverId)`event to mark that the server certificate is being used in the rest of the handshake process. Once validation is successful, the client proceeds to send its own certificate to the server, followed by a *CertificateVerify* and a *Finished* messages. These steps are cryptographically bound to the handshake transcript using hashing and signing functions, ensuring integrity and authenticity. At this stage, the client considers the session secure and triggers the `authSuccess(clientId)` event. If the server is successfully authenticated, the client encrypts and sends a secure payload using the server's public key and considers the handshake complete by triggering `completedHandshake` event, as follows:

```
...
event certValidated(serverId);
let serverPubKey = extractPubKeyFromCert(serverCert
    ) in
event receivedCert(serverId);
out(main_channel, certificateMsg(clientCert));
...
out(main_channel, certificateVerify(
    computeITSVerify( toTranscriptHash(tHash),
    clientPrivKey, clientPSID, tlsHandshake)));
out ( main_channel, finishedMsg ( sign ( hkdf ( tHash
    , client_finished_label), clientPrivKey)));
event authSuccess(clientId);
out(main_channel, aenc(secure_data, serverPubKey))
    ;
event completedHandshake
```

4.2.3. Server process

The server process modeled in ProVerif typically embodies an RSU that responds to a secure connection attempt initiated by a client (a vehicle). The process follows the steps defined in the TLS 1.3 protocol, with authentication using ITS certificates. Thus, after generating its ITS certificate, the server initiates the handshake process by receiving a *ClientHello* message from the client where the `beginHandshake(serverId)` event represents the start of the server-side authentication attempt, as follows:

```
let serverProcess =
...
let serverCert = generateITScert(serverId,
    serverPubKey, currentTime, certValidityPeriod,
    serverPerms, itsRootCA ) in
in(main_channel, clientHelloMsg: tlsMessage);
event beginHandshake(serverId);
```

In response, the server starts sending successively to the client the expected TLS 1.3 messages by specifying the agreement on authentication using the ITS certificate. Furthermore, the server constructs a cryptographic hash of the current handshake transcript, referred to as partial hash, using nested hash and then sends CertificateVerify and Finished messages, both signed using the server's private key. These messages are constructed using the constructors defined in ProVerif and sent on the channel *main_channel*, as follows:

```
out(main_channel, serverHello(serverNonce,
    client_cert_
type_its));
out(main_channel, encryptedExtensions(
    server_cert_type_
its));
out(main_channel, cert_request);
out(main_channel, certificateMsg(serverCert));
let partialHash = hash( ... );
out(main_channel, certificateVerify(
    computeITSVerify( toTranscriptHash(partialHash
    ), serverPrivKey, serverPSID, tlsHandshake)));
out(main_channel, finishedMsg(sign(hkdf(
    partialHash, server_finished_label),
    serverPrivKey)));
```

Following these messages, the server then receives the client's certificate and proceeds to verify that it is valid and correctly signed. If the certificate is indeed authentic, the events `certValidated (clientId)` and `receivedCert(clientId)` are triggered, as follows:

```
in(main_channel, clientMsg: tlsMessage);
let certificateMsg(clientCert) = clientMsg in
let certStart = extractTimestamp(clientCert) in
let certEnd = extractExpiration(clientCert) in
...
event certValidated(clientId);
let clientPubKey = extractPubKeyFromCert(clientCert
    ) in
event receivedCert(clientId);
```

As a final verification phase, the server authenticates the client by verifying its signature on the entire handshake transcript. This is done by checking whether the verification result using the client's public key matches the computed `fullHash`, which represents the hash of the complete handshake transcript. If this verification succeeds, the server emits `authSuccess(serverId)` event, confirming successful client authentication. Following this, the server proceeds to receive encrypted data through the main communication channel. The process concludes with the `completedHandshake event`, marking the successful completion of the ITS-TLS mutual authentication handshake protocol and the transition to secure data exchange:

```
if checksign(sigVal, clientPubKey) = fullHash then
event authSuccess(serverId);
in(main_channel, encData: bitstring);
let _ = adec(encData, serverPrivKey) in
event completedHandshake
```

4.3. Security properties

To assess the robustness of the authentication mechanisms in TLS 1.3 using ITS certificates, we formally specify and verify a set of key security properties using ProVerif. These properties check if the handshake achieves session key confidentiality, mutual authentication, certificate integrity validation, and context binding. This modeled in ProVerif using a set of queries under the Dolev–Yao attacker model.

4.3.1. Confidentiality of session keys

This propriety models the secrecy of the negotiated session key `psk`. Thus, if the query succeeds, it means that the attacker cannot compute or learn the session key from observing or interfering with the handshake. We formalize it in ProVerif as follows:

```
query attacker(psk).
```

4.3.2. Mutual authentication

This property verifies that when either party, a client or a server, reaches a state of successful authentication, the peer has also reached a corresponding authenticated state within the same handshake session. Formally, this property is expressed as correspondence queries ensuring that an *authSuccess* event on one side implies a matching *authSuccess* event on the other side, with injective correspondence to prevent replay attacks. This ensures that every completed handshake corresponds to a fresh and unique session involving the intended peer. This modeled in ProVerif as follows:

```
query client: identity, server: identity;
event(authSuccess(client))==>event(authSuccess(
    server)).
query client: identity, server: identity;
inj-event(authSuccess(server)) ==> inj-event(
    authSuccess (client)).
query id: identity;
inj-event(beginHandshake(id)) ==> inj-event(
    authSuccess(id)).
```

4.3.3. Validity and freshness of certificate

In TLS 1.3 handshake, both the client and the server require that the peer's certificate be successfully validated before completing authentication process. From the client's perspective, a handshake is only considered successful if the server's certificate has passed all the checks imposed by RFC 8902, including verification of its short validity period, permissions, signature by a trusted CA, and identity match. Similarly, from the server's perspective, authentication is only accepted if the client's certificate meets the same validation criteria. In our formal model, this is expressed in ProVerif by the following queries which ensure that the *authSuccess* event for either party can only occur if the corresponding *certValidated* event has occurred first:

```
query client: identity, server: identity;
event(authSuccess(client)) ==> event(certValidated
    (server )).
query client: identity, server: identity;
event(authSuccess(server)) ==> event(certValidated
    (client )).
```

4.3.4. Context binding for PduFunctionalType

The handshake context binding property ensures that in every successful TLS 1.3 handshake using ITS certificates, the *pduFunctionalType* filed included in the cryptographic computation is explicitly set to *tlsHandshake* value. This enforces a strict separation between handshake and application logic, and prevents attacks where valid handshake signatures could be replayed or misused in another functional context such as application data. In our model, this property is enforced by defining the `handshakeContext(id, tlsHandshake)` event, triggered when handshake verification is computed in both client and server processes. Thence, to ensure that any `authSuccess` event is preceded by a signature computation bound to the correct handshake context, we define the following queries:

```
query id: identity;
event(authSuccess(id)) ==> event(handshakeContext(
    id , tlsHandshake)).
query id:identity, pdu:pduFunctionalType;
event(handshakeContext(id, pdu)) ==> pdu =
    tlsHandshake.
query id: identity;
inj-event(authSuccess(id)) ==> inj-event(
    handshakeContext ( id, tlsHandshake)).
```

To model multiple concurrent protocol sessions, unlimited instances of client and server processes are run in parallel using the following ProVerif process:

```
process
  (!clientProcess) | (!serverProcess)
```

We present in Fig. 8 a detailed overview of the main exchanged messages and critical security events captured in our ProVerif ITS-TLS authentication model, thereby illustrating the protocol flow between the client and the server using ITS certificates.

5. Formal verification results

To verify whether the formal ITS-TLS protocol model satisfies the different security properties previously discussed, we use ProVerif 2.05[2,3] as a verification tool. The experiments were performed on a system running Windows 10 Professional with an Intel Core i5-3230M processor @ 2.60 GHz and 8.0 GB of RAM. The verification of the various properties was completed in approximately 165 ms.

5.1. Verification results

We present the formal verification results in Table 2, which summarizes the outcome of our ProVerif analysis. The results confirm that the essential security properties of the TLS 1.3 handshake with ITS certificates are preserved under the assumption that the communicating entities behave honestly. Specifically, secrecy of the pre-shared key (P1), successful mutual authentication between ITS client and server (P2), certificate validation (P4), permission verification through PSID and SSP values (P5), and the strict context binding to the tlsHandshake functional type (P6) were all successfully verified under the adversary's capabilities.

[2] https://bblanche.gitlabpages.inria.fr/proverif//
[3] http://proverif24.paris.inria.fr/index.php//

Table 2
Summary table of security properties verified with ProVerif.

	Property	Result
P1	Secrecy of the pre-shared key (psk) must be preserved under adversarial capabilities	True
P2	Mutual authentication must be successfully established between ITS client and server entities, and both parties verify each other's identity	True
P3	Every beginHandshake event between ITS entities must eventually result in a corresponding authSuccess event, ensuring successful session establishment	**False**
P4	Mutual certificate validation must ensure that successful authentication of either the client or server implies both parties have correctly checked each other's ITS certificates before completing the handshake	True
P5	Permission verification must ensure PSID and SSP values are strictly compliant with ITS-TLS handshake requirements after successful authentication	True
P6	Any handshake uses ITS certificates must ensure context binding, and the pduFunctionalType field must be accepted only if its value is tlsHandshake	True

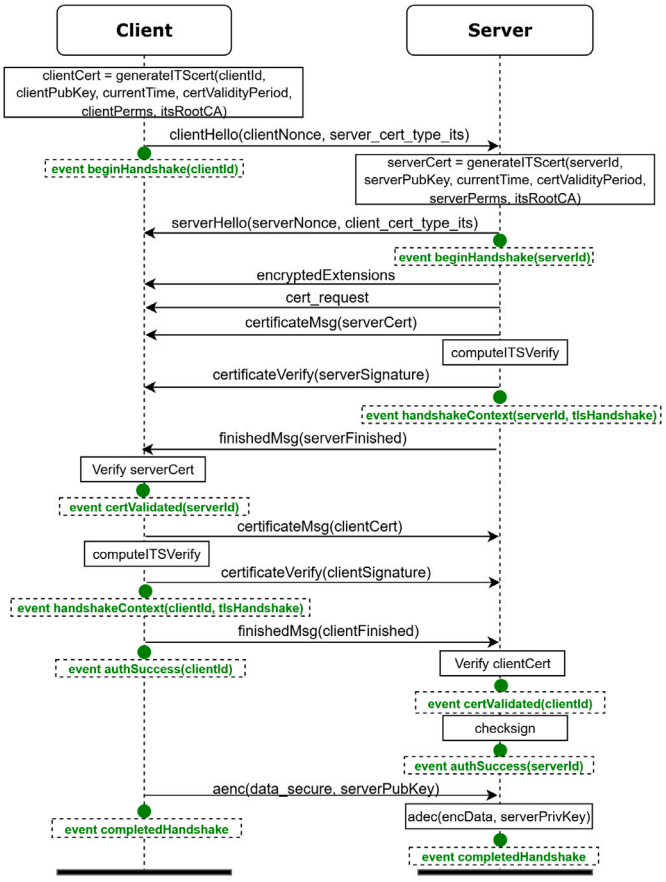

Fig. 8. Flow diagram of TLS 1.3 handshake using ITS certificates: the exchange steps between the client and the server, as well as critical ProVerif events.

In contrast, the property (P3), formulated as the requirement that each *beginHandshake* event must eventually lead to a corresponding *authSuccess* event, was found to be false. This result highlights that, while confidentiality and authenticity are ensured, availability is not guaranteed in the symbolic model. In particular, the handshake remains vulnerable to adversaries who can exploit the complex ITS certificate validation process to initiate sessions without completing them, thus preventing the handshake from reaching a successful state.

5.1.1. Violation of property P3

The property P3 requires that whenever an ITS client initiates a TLS 1.3 handshake with a legitimate server using ITS certificate, the protocol must guarantee progress towards successful session establishment. However, the complexity of the verification process makes

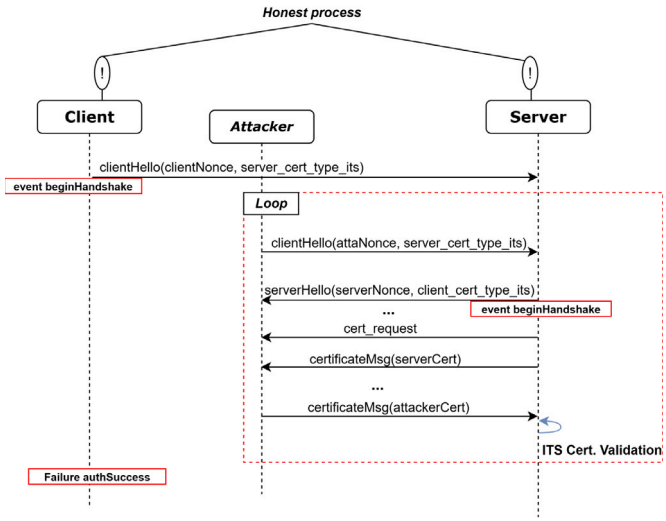

Fig. 9. Trace of Denial of Service attack on the property P3.

the handshake vulnerable to availability attacks. Thus, Fig. 9 shows a counterexample of a scenario where the attacker can exploit the nature of TLS certificates to overload the server and cause a denial of service, preventing honest clients from completing their handshake and reaching the *authSuccess* state. Here, in this scenario, the attacker repeats sending ClientHello messages to initiate multiple handshake requests in parallel. At each iteration, it generates a fresh nonce and an ITS certificate for its identity, and then sends a *ClientHello* message to the server. When the server responds with a *CertificateRequest*, the attacker returns a certificate via *certificateMsg(attackerCert)*, triggering expensive validation operations for each received certificate as detailed previously. By repeating this pattern indefinitely, the attacker floods the server with incomplete handshakes, saturating its resources and delaying or more preventing the completion of legitimate handshakes. Thus, while authentic clients believe their handshake will succeed (*event(authSuccess())*), the server is unable to process all requests, demonstrating the violation of the availability property. This is critical in ITS environments and can result in the loss or delay of important safety messages.

5.1.2. Potential vulnerabilities scenarios under weak ITS certificate verification

The security guarantees of ITS-TLS critically rely on the strict validation of ITS certificates, as previously described in Section 3.2.2. While our formal analysis results confirm that these checks provide strong authenticity and authorization guarantees under ideal conditions, any deficiency or omission of any part of the certificate validation steps could introduce significant risks. In practice, the weak enforcement of fields, such as validity periods (start and end), *psid* and *ssp* permissions,

Table 3
Potential security vulnerabilities due to improper validation of ITS certificates.

Weakness	Exploitation	Impact
Validity period not strictly enforced	Attacker can reuse an expired or revoked certificate	Compromised authenticity and acceptance of illegitimate peers
psid and ssp consistency not verified	Misuse of certificates for unauthorized services during handshake	Misuse of application permissions and privilege escalation
pduFunctionalType ≠ tlsHandshake accepted	Injection of SignedData intended for other PDUs into TLS flow	Semantic confusion and risk of data injection
No generationTime freshness check	Replay of old valid CertificateVerify messages	Loss of session freshness and enabling replay attacks
Incomplete certificate chain verification	Acceptance of certificates from rogue or misconfigured CAs	PKI compromise and acceptance of unauthorized entities

or *pduFunctionalType* value may open the way to adversarial exploitation. Thus, as summarized in Table 3, several scenarios illustrate how such weaknesses can materialize.

Weak or incomplete verification of the entire ITS certificate chain opens several specific attack vectors, directly related to the critical steps we modeled as shown in Fig. 10. In fact, expired certificate acceptance can occur if the verifier entity (specifically the server) does not rigorously verify the validity period, allowing an attacker to authenticate with outdated or revoked credentials. In more detail, this can occur when the server does not correctly evaluate timestamps via `extractTimestamp` and `extractExpiration` either by time synchronization inaccuracies or by malicious time-manipulation attacks, it can emit an `authSuccess()` event without `certValidated()` having been guaranteed. Similarly, the improper *pduFunctionalType* validation may permit *SignedData* structures intended for other applications to be injected into the TLS handshake, creating semantic confusion and weakening protocol guarantees. When this step is omitted, a *certificateVerify()* message carrying a *SignedData* intended for another use can still lead to `certValidated()`. In addition, the weak control of *headerInfo.psid* and associated permissions in `makePermissions(psid, ssp)` can compromise the authorization logic, which is of crucial importance in V2X communications as they depend on strict permissions to enable a specific service, such as in DENM/CAM. Furthermore, weak or ignored *generationTime* checks make the protocol vulnerable to replay attacks in CertificateVerify, where adversaries reuse previously valid handshake messages specifically in high-mobility scenarios. These weaknesses show that in our model the `authSuccess()` event could occur without the expected preconditions (`certValidated()`, `handshakeContext()`), which concretely reflects the impact of incomplete verification of ITS certificates. These scenarios lead to cross-consequences ranging from authenticity breach, partial availability loss (rejection of legitimate clients), authorization abuse, and traceability risks.

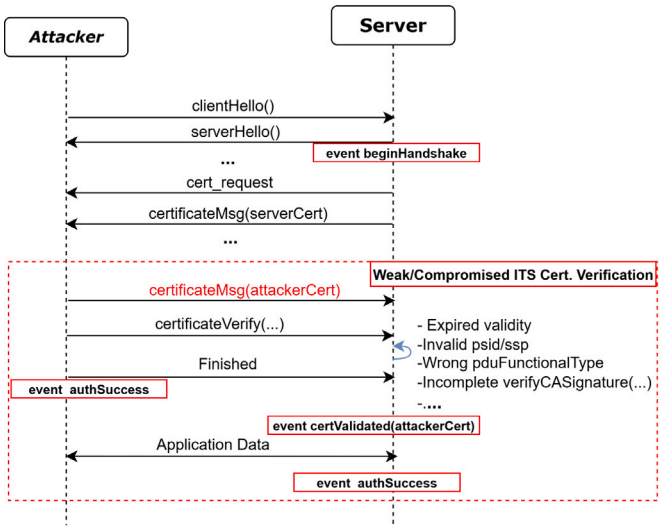

Fig. 10. Message flow illustrating how weak or omitted ITS certificate validation allows a forged or compromised attacker's certificateMsg/certificateVerify to be accepted and cause certValidated and authSuccess to occur on the server side.

5.2. Analysis of interoperability risks in ITS-TLS integration

Beyond the results of the formal analysis, the practical usage of ITS certificates within the TLS 1.3 handshake raises a number of interoperability issues that can directly affect protocol reliability in real-world implementations. In this section, we discuss the key interoperability challenges identified through our analysis, focusing on the fundamental inconsistencies between IEEE 1609.2 and traditional X.509 certificate frameworks. We summarize the identified risks to interoperability in Table 4.

The successful real-world implementation depends on deep modifications to both cryptographic libraries and PKI management logic in TLS 1.3. Hence, ensuring consistent validation and permission semantics across heterogeneous vehicular systems remains a persistent challenge for correct ITS-TLS adoption. Thus, unlike the ASN.1/DER-encoded X.509 format supported by conventional TLS libraries, ITS certificates rely on OER encoding and include additional attributes that define application permissions and context. These structural changes imply that standard parsing and verification functions in existing TLS stacks (e.g. OpenSSL, GnuTLS, etc.) (Rasoamanana et al., 2022) cannot directly process ITS credentials without additional adaptations. Moreover, the ITS PKI model diverges from the X.509 hierarchical trust structure by introducing distinct enrollment and authorization authorities (EA/AA). This architectural difference prevents direct application of classical TLS chain validation and revocation mechanisms, which must be redesigned to accommodate ITS-specific authorization semantics. In practice, ITS certificate revocation employs a dual approach in which long-term enrollment certificates are revoked via conventional CRLs, while short-lived authorization tickets rely on their inherently limited validity period as the primary revocation mechanism. For TLS integration, implementations must therefore support CRL Series fetching as defined in ETSI TS 102 941, maintain accurate time synchronization for expiration validation, and implement secure online repository queries when certificate chains are incomplete. These mechanisms, though optimized for V2X broadcast latency requirements, introduce additional complexity for TLS stacks that must integrate ITS-specific revocation workflows alongside standard validation logic. Hence, cross-domain trust translation becomes critical to maintain consistent security policies in hybrid handshake mixing X.509 and ITS entities.

Another source of potential interoperability issues lies in the handshake negotiation process and cryptographic compatibility. As the new included certificate_type extensions must be consistently negotiated by both peers, session establishment may fail due to unknown certificate type and signature verification errors in the Certificates and CertificateVerify messages when a peer does not support or misinterprets these extensions. Furthermore, as mentioned in Section 3.3,

Table 4
Identified interoperability challenges and their impact on ITS-TLS integration.

Challenge category	Issue source	Interoperability risk
Certificate encoding format	OER encoding vs ASN.1/DER structure	Parsing or decoding failure with standard TLS libraries
Extensions negotiation	Unsupported certificate_type 1609Dot2 extension	TLS 1.3 message parsing failed
PKI model	ITS-specific authorities(EA[a]/AA[b]) vs X.509 hierarchy	Inaccurate trust evaluation
Validity/Permission semantics	Different validation logic (psid, ssp, generationTime)	Inconsistent validation leading to untrusted sessions
Cryptographic Algorithms	ITS signature schemes vs standard TLS algorithms	Signature verification incompatibility
Hybrid exchange	Mixed ITS and X.509 nodes	Cross-domain authentication failure

[a] Enrollment Authority.
[b] Authorization Authority.

ITS certificates rely on ECC and signature algorithms that are not supported by current TLS 1.3 implementations with X509 certificates. This requires an additional cryptographic stack to support these two different formats and achieve interoperability, especially in the case of mixed exchanges using different types of certificates. Addressing these interoperability issues requires not only protocol-level adaptation, but also coordinated evolution of cryptographic libraries, PKI management strategies, and application layer validation logic to ensure reliable and secure adoption.

5.3. Lessons learned and recommendations

From this analysis we can draw two essential lessons. First, the main potential vulnerabilities for ITS-TLS are not cryptographic, but semantic arising from the complexity of the integration between these two different ecosystems. Thus, the strict validation of the *HeaderInfo* fields, particularly the *pduFunctionalType* and *psid/ssp* permissions, is critical and any omission can lead to severe context confusion and privilege escalation attacks. Secondly, the increased complexity of the protocol, due to the use of OER encoding with the enriched ITS certificate semantics, significantly expands the attack surface and introduces new threats, such as parser-induced denial of service. Thus, transforming the simple TLS *CertificateVerify* message into a complex IEEE 1609.2 SignedData structure encoded in OER can potentially increase security attacks and the risks of poor implementation of the complex validation sequence, leading to serious vulnerabilities such as permission bypass. These results confirm the soundness of the design at the protocol level, but also motivate complementary experimental studies and realistic simulations to validate the protocol in realistic scenarios.

Based on these findings, we emphasize as recommendations to strengthen the resilience of ITS-TLS, that future works should combine symbolic analysis with experimental evaluations, particularly to measure the impact of mobility on handshake latency and ITS certificate validation performance in various mobility conditions, as network disruption or handover events in real vehicular environments could severely impact the reliability of handshake. Furthermore, future directions should focus on how to design lightweight defense strategies that help reduce the impact of repeated handshake attempts and mitigate resource exhaustion risks, such as rate-limiting techniques (Tiloca et al., 2017). Furthermore, as the validity period of ITS certificates is short, and their verification is a crucial step in ensuring proper authentication, robust time validation mechanisms, such as using multi-source time synchronization, are essential to mitigate time-shifting attacks. Finally, we also encourage standard bodies to mandate these best practices in future revisions and explore privacy-enhancing techniques to prevent vehicle tracking based on exposed certificate data during the exchange of handshake messages.

6. Experimental evaluation

To complement the formal analysis, we conduct a simulation-based experimental study comparing classical TLS using X.509 certificates with TLS incorporating ETSI ITS certificates. The objective is to quantify how differences in structure and validation semantics affect handshake latency and certificate validation performance.

6.1. ETSI ITS certificate generation

To ensure correctness and compliance with standard specifications, the ITS certificates used in our evaluation are generated in accordance with ETSI TS 103 097 V2.2.1 (ETSI, 2026), which defines the certificate structure, mandatory fields, and encoding rules for ITS security. The implementation follows an ASN.1-based certificate model adapted from Hammi et al. (2017), aligned with the requirements of the latest specification and serving as a practical implementation baseline.

The certificate generation process incorporates the core elements defined in the standard, including: subject information specifying the certificate type (e.g., authorization ticket or enrollment credential) and subject identifier, subject attributes comprising the verification public key, application permissions represented as PSID/SSP pairs and the assurance level, and validity constraints expressed as a start time and expiration. Additionally, optional fields such as geographic region restrictions are included to reflect realistic ITS deployment scenarios.

For encoding, certificates are serialized using an OER-like binary representation as recommended by ETSI TS 103 097. This enables efficient encoding and compact representation while preserving structural compatibility with the ASN.1. Furthermore, to ensure representativeness, certificate fields are randomized across iterations: subject identifiers are generated dynamically, PSID values are selected from valid ITS application ranges, and SSP contents are randomly instantiated within valid size constraints. This approach enables the evaluation to capture variability in certificate structure and supports a realistic assessment of performance metrics.

6.2. Experimental setup

For our experiments, we used a machine equipped with an Intel Core i5-3230M processor @ 2.60 GHz and 8.0 GB of RAM. The benchmarking framework was implemented in Python 3.14 using the cryptography library (version 44.0.2) for cryptographic operations and certificate generation.

For fairness, both certificate types rely on the same cryptographic primitives, ECDSA over the NIST P-256 curve with SHA-256, with a total of 1000 certificates generated for each format. The evaluation measures: (i) **certificate encoding time** (OER for ITS and DER for X.509), (ii) **ECDSA signature verification time**, (iii) **ITS-specific authorization overhead** (PSID/SSP validation), and (iv) **certificate size**. The total validation time is computed as the sum of encoding and cryptographic verification operations.

6.3. Results and discussion

The experimental results for ITS-TLS integration performance are summarized in Table 5. As shown, ITS certificate validation with 0.1843 ms is nearly identical to that of X.509 ECC with 0.1829 ms, representing a negligible overhead of less than 1%. This marginal difference stems primarily from the ITS-specific PSID/SSP authorization step (0.0011 ms). In addition, ITS OER encoding with 0.0073 ms outperforms X.509 DER encoding (0.0097 ms), reflecting the compact

Table 5
Performance comparison between ETSI TS 103 097 and X.509 ECC certificates. Values are presented as mean ± standard deviation ($n = 15$).

Metrics	ITS (ETSI)	X.509 ECC
Encoding Time (ms)	0.0073 ± 0.0005	0.0097 ± 0.0003
ECDSA Verification (ms)	0.1759 ± 0.0024	0.1731 ± 0.0031
PSID/SSP (ms)	0.0011	N/A
Certificate Size (bytes)	123.50 ± 0.24	388.00 ± 0.02
Total Validation (ms)	0.1843 ± 0.0025	0.1829 ± 0.0029

binary format and reduced structural overhead of ITS certificates. As expected, ECDSA verification dominates total validation time in both schemes, with comparable performance due to the shared P-256 curve. A key distinction lies in certificate size, where ITS certificates (123.5 bytes) are approximately three times smaller than X.509 (388.0 bytes), a reduction that directly benefits bandwidth-constrained V2X communications.

These preliminary results demonstrate that ITS certificates achieve computational performance comparable to X.509 ECC while delivering substantial size efficiency with a reasonable validation overhead. These findings confirm the practical viability of ITS-TLS, while further investigation is required to assess performance under real-world mobility conditions and resource-constrained onboard units.

7. Discussion

This section provides a broader perspective for our work addressing the scope of the symbolic model, the emerging threat of quantum computing for TLS deployments, and alternative protocols for securing vehicular communications.

7.1. Limitations of the symbolic model

To the best of our knowledge, our presented work constitutes the first formal verification effort related to the integration of TLS 1.3 with ITS certificates as specified in RFC 8902. Through the use of ProVerif tool, we demonstrated that fundamental security properties are guaranteed when implemented adequately. However, we also emphasize that the presented model abstracts low-level implementation details and physical layer characteristics. Thus, although the symbolic verification provides strong guarantees regarding protocol logic and message flows, it relies on the Dolev–Yao adversary model, which assumes perfect cryptographic primitives. While this assumption is standard in formal verification, it does not capture certain practical threats that may arise in real-world deployments. In particular, the symbolic model does not account for implementation-level vulnerabilities such as side-channel attacks, timing leaks, or flaws in cryptographic libraries that may compromise security in deployed systems. Consequently, while symbolic verification offers a strong fundamental guarantee, it requires further validation through real-world simulations and testing to fully capture the complexities of actual deployments.

7.2. Post-quantum transition in TLS

While TLS 1.3 currently provides strong security using classical asymmetric primitives, ECDH (e.g., X25519, P-256) for key exchange and RSA/ECDSA for certificate signatures, the rapid development of quantum computing necessitates a new assessment of these guarantees. Thus, Shor's algorithm (Shor, 1994), enabling "harvest-now, decrypt-later" attacks, can compromise the asymmetric primitives of TLS 1.3 undermining forward secrecy and authentication, while symmetric algorithms such as AES and ChaCha20 remain secure provided that larger key sizes are used to maintain quantum resistance. For this, to address the evolving post-quantum cryptography (PQC) threats, the IETF, NIST, and major library projects are actively standardizing and deploying post-quantum Key Encapsulation Mechanisms (KEMs) such as ML-KEM (formerly CRYSTALS-Kyber, NIST FIPS-203) and post-quantum signature schemes like ML-DSA (formerly CRYSTALS-Dilithium, NIST FIPS-204) and SLH-DSA (FIPS-205), which replace or augment classical primitives in the TLS handshake (National Institute of Standards and Technology (NIST), 2024). Furthermore, the IETF's latest guidance recommends the adoption of hybrid key exchange schemes of TLS 1.3, such as X25519MLKEM768, to protect against both classical and quantum attacks, rather than relying solely on classical or purely post-quantum groups (Stebila et al., 2025).

In practice, major implementations (e.g., OpenSSL, BoringSSL, Cloudflare's TLS stacks) are experimenting with hybrid key exchange constructions, combining classical X25519 with PQC KEMs and dual public-key certificate formats that carry both classical and PQC keys/signatures, enabling backward compatibility during transition while providing quantum resistance. These adaptations impact core TLS 1.3 handshake elements such as the ClientHello key_share and supported_groups extensions, the ServerHello key_share selection and certificate_request structures, and certificate chain encoding, increasing handshake size and complexity (Montenegro et al., 2025). Consequently, formal analysis must evolve to model hybrid negotiations and PQC primitives accurately, ensuring that security proofs remain valid in the presence of both classical and emerging quantum-resistant cryptographic schemes.

7.3. Alternative secure communication protocols

Beyond TLS, several alternative protocols may also be considered for securing vehicular communications, depending on the communication requirements. In particular, Datagram Transport Layer Security (DTLS) 1.3 (Kothmayr et al., 2012) extends the TLS security architecture to datagram-based communication over UDP, making it particularly suitable for latency-sensitive V2X applications where the overhead of TCP's reliable delivery is undesirable. Another emerging transport protocols is Quick UDP Internet Connections (QUIC) (Kumar and Dezfouli, 2025), which integrates TLS 1.3 security mechanisms directly within the transport layer while supporting multiplexed streams and reduced connection establishment latency through features such as 0-RTT session resumption. These characteristics may be beneficial in highly dynamic ITS environments where connections are frequently established and mobility is high. In parallel, the vehicular ecosystem also relies on dedicated security frameworks defined by the standards IEEE 1609.2 and ETSI TS 103 097, which provide lightweight security mechanisms specifically designed for V2X broadcast messages. The coexistence of these protocols suggests that future ITS architectures will adopt a hybrid approach, combining TLS/DTLS for cloud services, QUIC for low-latency connections, and native ITS security for safety-critical broadcast messages, each selected according to specific application requirements and operational constraints.

8. Conclusion

In this work, we presented the first formal verification of ITS-TLS, which integrates TLS 1.3 with IEEE 1609.2 ITS certificates for vehicular communication. Using the applied pi-calculus and the ProVerif model checker, we propose a formal model of the TLS handshake that integrates ITS certificates, addressing both the certificate exchange process and the validation of ITS-specific security constraints. During this work, we formally analyzed critical properties such as secrecy of the session key, mutual authentication, certificate validation correctness, and context binding. The analysis confirms that, under symbolic assumptions and honest participant behavior, the protocol satisfies its intended security guarantees. At the same time, our work revealed that the complexity of ITS certificate verification introduces protocol-level vulnerabilities that can be exploited even when the implementation is correct. In particular, we highlighted availability risks due to the

computational cost of certificate parsing and verification. These insights demonstrate that while TLS 1.3 provides strong cryptographic foundations, its integration with ITS certificates introduces unique attack surfaces that are not present in its conventional deployments.

The benchmark evaluation of ITS certificates against standard X.509 further demonstrates their performance suitability and underscores the relevance of ITS-TLS integration for V2X environments. By achieving comparable validation latency with a significant reduction in certificate size, ITS certificates offer a compelling trade-off between computational overhead and communication efficiency. Consequently, these findings support the feasibility of adopting ITS certificates within TLS 1.3 as a scalable and efficient security mechanism for large-scale ITS deployments.

This study represents an advance in the formal understanding of the security properties of ITS-TLS and comprehensively discusses the fundamental aspects of this integration. Nevertheless, using symbolic model, assuming perfect cryptography, cannot fully capture the strict real-time constraints of dynamic V2X environments, nor the complex privacy aspects of vehicular communications. As future work, we plan to extend the verification towards computational models, integrate realistic mobility and extended performance simulations, and investigate practical countermeasures for the identified vulnerabilities. This will help to ensure that ITS-TLS can withstand both traditional threats and ITS-specific vulnerabilities in real-world deployments. Finally, by providing a consolidated view of ITS-TLS security issues, we hope that this work will serve as a basis for researchers interested in addressing the challenges and research opportunities identified in this study.

CRediT authorship contribution statement

Sihem Baccari: Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis. **Mohamed Hadded:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Funding acquisition, Formal analysis.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Mohamed Hadded reports financial support was provided by Abu Dhabi University. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Abu Dhabi University Office of Research and Sponsored Programs.

Data availability

No data was used for the research described in the article.

References

3GPP, 2020. TS 22.186 V16.2.0, service requirements for enhanced V2X scenarios (release 16). URL https://www.etsi.org/deliver/etsi_ts/122100_122199/122186/16.02.00_60/ts_122186v160200p.pdf?.

3GPP, 2025a. 3GPP release 17. URL https://www.3gpp.org/specifications-technologies/releases/release-17. (Accessed 9 September 2025).

3GPP, 2025b. 3GPP release 18. URL https://www.3gpp.org/specifications-technologies/releases/release-18. (Accessed 9 September 2025).

(5GAA), G.A.A., 2018. V2X functional and performance test report. URL https://5gaa.org/content/uploads/2018/11/5GAA_P-190033_V2X-Functional-and-Performance-Test-Report_final-1.pdf?.

(5GAA), A.A., 2023. C-V2X use cases. Requirements and service level agreements. URL https://5gaa.org/content/uploads/2023/02/5gaa-t-200111-tr-c-v2x-use-cases-and-service-level-requirements-vol-i-v3.0-clean-version-1.pdf?.

802.11p, 2010. IEEE standard for information technology - telecommunications and information exchange between systems - local and metropolitan area networks - specific requirements part 11. Wirel. LAN Mediu. Access Control. (MAC) Phys. Layer (PHY) Phys. Layer (PHY) Specif. Amend. 6.

Abadi, M., Blanchet, B., Fournet, C., 2018. The applied pi calculus: Mobile values, new names, and secure communication. J. ACM 65 (1), 1–41. http://dx.doi.org/10.1145/3127586.

Abdelhafez, M.E., Ramadass, S., Abdelwahab, M., 2023. TLS guard for TLS 1.3 zero round-trip time (0-RTT) in a distributed environment. J. King Saud Univ. - Comput. Inf. Sci. 35 (10), 101797.

Adja, Y.C.E., Hammi, B., Serhrouchni, A., et al., 2021. A blockchain-based certificate revocation management and status verification system. Comput. Secur. 104, 102209.

Ali, M., Kim, S.M., Kim, J., 2024. Enhanced C-V2X mode-3 for CAM and DENM propagation in scalable overlapped enbs. In: 2024 15th International Conference on Information and Communication Technology Convergence (ICTC), Jeju Island, Korea, Republic of. pp. 1231–1232. http://dx.doi.org/10.1109/ICTC62082.2024.10827152.

Alliance, N.G.M.N.N., 2018. V2X white paper. URL https://5gaa.org/content/uploads/2018/08/V2X_white_paper_v1_0.pdf, V2X White Paper.

Baccari, S., Hadded, M., Touati, H., Ghazzai, H., Laouiti, A., Setti, G., 2025. Multi-technology integration in hybrid V2X networks: A survey, challenges, and open issues. IEEE Access 13, 138475–138508. http://dx.doi.org/10.1109/ACCESS.2025.3595461.

Berbecaru, D.G., Lioy, A., 2023. An evaluation of X.509 certificate revocation and related privacy issues in the web PKI ecosystem. IEEE Access 11, 79156–79175. http://dx.doi.org/10.1109/ACCESS.2023.3299357.

Bhargavan, K., Blanchet, B., Kobeissi, N., 2017. Verified models and reference implementations for the TLS 1.3 standard candidate. In: 2017 IEEE Symposium on Security and Privacy. SP, pp. 483–502. http://dx.doi.org/10.1109/SP.2017.26.

Blanchet, B., 2001. An efficient cryptographic protocol verifier based on prolog rules. In: Proceedings. 14th IEEE Computer Security Foundations Workshop, 2001., Cape Breton, NS, Canada. pp. 82–96. http://dx.doi.org/10.1109/CSFW.2001.930138.

Blanchet, B., 2016. Modeling and verifying security protocols with the applied pi calculus and ProVerif. Found. Trends Priv. Secur. 1 (1–2), 1–135. http://dx.doi.org/10.1561/3300000004.

Blanchet, B., Cheval, V., Cortier, V., 2022. ProVerif with lemmas, induction, fast subsumption, and much more. In: 2022 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA. pp. 69–86. http://dx.doi.org/10.1109/SP46214.2022.9833653.

Blanchet, B., Smyth, B., Cheval, V., et al., 2018. ProVerif 2.00: Automatic cryptographic protocol verifier, user manual and tutorial. Version from 16, 05–16.

Boulila, N., Hadded, M., Laouiti, A., Azouz Saidane, L., 2018. QCH-MAC: A qos-aware centralized hybrid MAC protocol for vehicular ad hoc networks. In: 2018 IEEE 32nd International Conference on Advanced Information Networking and Applications. AINA, pp. 55–62. http://dx.doi.org/10.1109/AINA.2018.00021.

Brecht, B., Therriault, D., Weimerskirch, A., Whyte, W., Kumar, V., Hehn, T., Goudy, R., 2018. A security credential management system for V2X communications. IEEE Trans. Intell. Transp. Syst. 19 (12), 3850–3871. http://dx.doi.org/10.1109/TITS.2018.2797529.

Bussa, S., Sisto, R., Valenza, F., 2025. Formal verification of a V2X scheme mixing traditional PKI and group signatures. J. Inf. Secur. Appl. 89, 103998. http://dx.doi.org/10.1016/j.jisa.2025.103998.

Chen, A.C.H., 2023. Evaluation and analysis of standard security techniques in V2x communication: Exploring the cracking of ECQV implicit certificates. In: 2023 IEEE International Conference on Machine Learning and Applied Network Technologies. ICMLANT, San Salvador, El Salvador, pp. 1–5. http://dx.doi.org/10.1109/ICMLANT59547.2023.10372987.

Chen, A.C.H., Lin, B.-Y., Liu, C.-K., Lin, C.-F., 2023. Implementation and performance analysis of security credential management system based on IEEE 1609.2 and 1609.2.1 standards. In: 2023 IEEE International Conference on Machine Learning and Applied Network Technologies. ICMLANT, San Salvador, El Salvador, pp. 1–5. http://dx.doi.org/10.1109/ICMLANT59547.2023.10372990.

Chen, A.C.H., Liu, C.-K., Lin, C.-F., Lin, B.-Y., 2024. V2X credential management system comparison based on IEEE 1609.2.1 and ETSI TS 102 941. In: 2024 IEEE North Karnataka Subsection Flagship International Conference. NKCon, Bagalkote, India, pp. 1–6. http://dx.doi.org/10.1109/NKCon62728.2024.10774595.

Clancy, J., et al., 2024. Wireless access for V2X communications: Research, challenges and opportunities. IEEE Commun. Surv. Tutor. 26 (3), 2082–2119. http://dx.doi.org/10.1109/COMST.2024.3384132, thirdquarter.

Cremers, C., Horvat, M., Hoyland, J., Scott, S., Van Der Merwe, T., 2017. A comprehensive symbolic analysis of TLS 1.3. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 1773–1788. http://dx.doi.org/10.1145/3133956.3134063.

Cremers, C., Horvat, M., Scott, S., Van Der Merwe, T., 2016. Automated analysis and verification of TLS 1.3: 0-RTT, resumption and delayed authentication. In: 2016 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 470–485. http://dx.doi.org/10.1109/SP.2016.35.

Dierks, T., Allen, C., 1999. The TLS protocol version 1.0. URL https://www.rfc-editor.org/info/rfc2246, RFC 2246.

Dierks, T., Rescorla, E., 2006. The transport layer security (TLS) protocol version 1.1. URL https://www.rfc-editor.org/info/rfc4346, RFC 4346.

Dierks, T., Rescorla, E., 2008. The transport layer security (TLS) protocol version 1.2. URL https://www.rfc-editor.org/info/rfc5246, RFC 5246.

Dolev, D., Yao, A., 1983. On the security of public key protocols. IEEE Trans. Inform. Theory 29 (2), 198–208. http://dx.doi.org/10.1109/TIT.1983.1056650.

Dowling, B., Fischlin, M., Günther, F., Stebila, D., 2021. A cryptographic analysis of the TLS 1.3 handshake protocol. J. Cryptology 34 (4), 37. http://dx.doi.org/10.1007/s00145-021-09384-1.

Eldewahi, A.E.W., Sharfi, T.M.H., Mansor, A.A., Mohamed, N.A.F., Alwahbani, S.M.H., 2015. SSL/TLS attacks: Analysis and evaluation. In: 2015 International Conference on Computing, Control, Networking, Electronics and Embedded Systems Engineering. ICCNEEE, Khartoum, Sudan, pp. 203–208. http://dx.doi.org/10.1109/ICCNEEE.2015.7381362.

ETSI, 2017. Intelligent transport systems (ITS); security; security header and certificate formats. ETSI TS 103 (097), V1, 3.1.

ETSI, 2018. Intelligent transport systems (ITS); security; trust and privacy management. ETSI TS 102, 941.

ETSI, 2019. TR 103 562 V2.1.1, intelligent transport systems (ITS); vehicular communications; basic set of applications; analysis of the collective perception service (CPS). URL https://www.etsi.org/deliver/etsi_tr/103500_103599/103562/02.01.01_60/tr_103562v020101p.pdf?.

ETSI, 2026. Intelligent Transport Systems (ITS); Security; Security Header and Certificate Formats; Release 2. Technical Report ETSI TS 103 097 V2.2.1, European Telecommunications Standards Institute, URL https://www.etsi.org/deliver/etsi_ts/103000_103099/103097/02.02.01_60/ts_103097v020201p.pdf.

Farran, H., Khoury, D., 2023. Performance improvements of vehicular PKI protocol for the security of V2X communications. In: 2023 46th International Conference on Telecommunications and Signal Processing. TSP, Prague, Czech Republic, pp. 177–182. http://dx.doi.org/10.1109/TSP59544.2023.

Hadded, M., Merdrignac, P., Duhamel, S., Shagdar, O., 2020. Security attacks impact for collective perception based roadside assistance: A study of a highway on-ramp merging case. In: 2020 International Wireless Communications and Mobile Computing. IWCMC, pp. 1284–1289. http://dx.doi.org/10.1109/IWCMC48107.2020.9148235.

Hadded, M., Muhlethaler, P., Laouiti, A., 2017. Performance evaluation of a TDMA-based multi-hop communication scheme for reliable delivery of warning messages in vehicular networks. In: 2017 13th International Wireless Communications and Mobile Computing Conference. IWCMC, pp. 1029–1034. http://dx.doi.org/10.1109/IWCMC.2017.7986427.

Hadded, M., Shagdar, O., Merdrignac, P., 2019. Augmented perception by V2X cooperation (PAC-V2X): Security issues and misbehavior detection solutions. In: 2019 15th International Wireless Communications & Mobile Computing Conference. IWCMC, pp. 907–912. http://dx.doi.org/10.1109/IWCMC.2019.8766683.

Hammi, B., Monteuuis, J.P., Daniel, E.S., Labiod, H., 2017. ASN.1 specification for ETSI certificates and encoding performance study. In: 2017 18th IEEE International Conference on Mobile Data Management. MDM, pp. 291–298. http://dx.doi.org/10.1109/MDM.2017.47.

IEEE, 2016. IEEE standard for wireless access in vehicular environments–security services for applications and management messages. In: IEEE Std 1609.2-2016 (Revision IEEE Std 1609.2-2013). pp. 1–240. http://dx.doi.org/10.1109/IEEESTD.2016.7426684.

ISO, 2024. Intelligent transport systems – ITS station security services for secure session establishment and authentication between trusted devices. (ISO 21177:2024), URL https://www.iso.org/obp/ui/en/#iso:std:iso:21177:ed-2:v1:en.

Kothmayr, T., Schmitt, C., Hu, W., Brünig, M., Carle, G., 2012. A DTLS based end-to-end security architecture for the internet of things with two-way authentication. In: 37th Annual IEEE Conference on Local Computer Networks - Workshops. pp. 956–963. http://dx.doi.org/10.1109/LCNW.2012.6424088.

Krawczyk, H., Eronen, P., 2010. HMAC-based extract-and-expand key derivation function (HKDF). URL https://www.rfc-editor.org/info/rfc5869, RFC 5869.

Krawczyk, H., Wee, H., 2016. The OPTLS protocol and TLS 1.3. In: 2016 IEEE European Symposium on Security and Privacy. EuroS&P, Saarbruecken, Germany, pp. 81–96. http://dx.doi.org/10.1109/EuroSP.2016.18.

Kumar, P., Dezfouli, B., 2025. kQUIC: A kernel-based quick UDP internet connections (QUIC) transport for IoT. IEEE Access 13, 100392–100404. http://dx.doi.org/10.1109/ACCESS.2025.3578120.

Lafourcade, P., Puys, M., 2016. Performance evaluations of cryptographic protocols verification tools dealing with algebraic properties. In: Foundations and Practice of Security. Springer, Cham, Switzerland, pp. 137–155. http://dx.doi.org/10.1007/978-3-319-30303-1_9.

Lee, H., Kim, D., Kwon, Y., 2021. TLS 1.3 in practice: How TLS 1.3 contributes to the internet. In: Proc. Web Conf. 2021. pp. 70–79.

Lekidis, A., Morais, H., 2024. Open V2X management platform cyber-resilience and data privacy mechanisms. In: Proceedings of the 19th International Conference on Availability, Reliability and Security. pp. 1–8.

Li, X., Xu, J., Zhang, Z., Feng, D., Hu, H., 2016. Multiple handshakes security of TLS 1.3 candidates. In: 2016 IEEE Symposium on Security and Privacy (SP), San Jose, CA, USA. pp. 486–505. http://dx.doi.org/10.1109/SP.2016.36.

Montenegro, J.A., Rios, R., Lopez-Cerezo, J., 2025. A performance evaluation framework for post-quantum TLS. Future Gener. Comput. Syst. 108062.

Msahli, M., Cam-Winget, N., Whyte, W., Serhrouchni, A., Labiod, H., 2020. TLS authentication using intelligent transport system (ITS) certificates. URL https://datatracker.ietf.org/doc/html/rfc8902, RFC 8902.

National Institute of Standards and Technology (NIST), 2024. Post-quantum cryptography. https://csrc.nist.gov/projects/post-quantum-cryptography. (Accessed 15 March 2026).

Rasoamanana, A.T., Levillain, O., Debar, H., 2022. Towards a systematic and automatic use of state machine inference to uncover security flaws and fingerprint TLS stacks. In: European Symposium on Research in Computer Security. Springer, pp. 637–657.

Rescorla, E., 2018. The transport layer security (TLS) protocol version 1.3. URL https://www.rfc-editor.org/info/rfc8446, RFC 8446.

Santesson, S., Myers, M., Ankney, R., Malpani, A., Galperin, S., Adams, D.C., 2013. X.509 internet public key infrastructure online certificate status protocol - OCSP. RFC 6960.

Sardar, M.U., Niemi, A., Tschofenig, H., Fossati, T., 2024. Towards validation of TLS 1.3 formal model and vulnerabilities in intel's RA-TLS protocol. IEEE Access 12, 173670–173685. http://dx.doi.org/10.1109/ACCESS.2024.3497184.

Satapathy, A., Livingston, J., et al., 2016. A comprehensive survey on SSL/TLS and their vulnerabilities. Int. J. Comput. Appl. 153 (5), 31–38.

Sedar, R., Kalalas, C., Vázquez-Gallego, F., Alonso, L., Alonso-Zarate, J., 2023. A comprehensive survey of V2X cybersecurity mechanisms and future research paths. IEEE Open J. Commun. Soc. 4, 325–391. http://dx.doi.org/10.1109/OJCOMS.2023.3239115.

Shor, P., 1994. Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings 35th Annual Symposium on Foundations of Computer Science. pp. 124–134. http://dx.doi.org/10.1109/SFCS.1994.365700.

Sirohi, P., Agarwal, A., Tyagi, S., 2016. A comprehensive study on security attacks on SSL/TLS protocol. In: 2016 2nd International Conference on Next Generation Computing Technologies. NGCT, Dehradun, India, pp. 893–898. http://dx.doi.org/10.1109/NGCT.2016.7877537.

Stebila, D., Fluhrer, S., Gueron, S., 2025. Hybrid key exchange in TLS 1.3. Internet-Draft draft-ietf-tls-hybrid-design-16, Internet Engineering Task Force, URL https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/16/, Work in Progress.

Tiloca, M., Gehrmann, C., Seitz, L., 2017. On improving resistance to denial of service and key provisioning scalability of the DTLS handshake. Int. J. Inf. Secur. 16 (2), 173–193.

Wouters, P., Tschofenig, H., Gilmore, J.I., Weiler, S., Kivinen, T., 2014. Using raw public keys in transport layer security (TLS) and datagram transport layer security (DTLS). URL https://www.rfc-editor.org/info/rfc7250, RFC 7250.

Yoshizawa, T., Singelée, D., Muehlberg, J.T., Delbruel, S., Taherkordi, A., Hughes, D., Preneel, B., 2023. A survey of security and privacy issues in v2x communication systems. ACM Comput. Surv. 55 (9), 1–36. http://dx.doi.org/10.1145/3558052.

Zhang, J., Yang, L., Cao, W., Wang, Q., 2020. Formal analysis of 5G EAP-tls authentication protocol using proverif. IEEE Access 8, 23674–23688. http://dx.doi.org/10.1109/ACCESS.2020.2969474.

Zhou, J., Fu, W., Hu, W., et al., 2024. Challenges and advances in analyzing TLS 1.3-encrypted traffic: A comprehensive survey. Electronics 13 (20), 4000. http://dx.doi.org/10.3390/electronics13204000.

Sihem Baccari received her Bachelor's degree in Computer Sciences in 2018 and the Research Master's degree in Computer Sciences and Networks, in December 2020, from the Faculty of Sciences of the University of Gabes, Tunisia. From June 2021 until now, Sihem Baccari holds the position of a blockchain developer at T+ company, France. Also, she is currently working as a research consultant in collaboration with Abu Dhabi University. Her research interests focuses on several issues related to Vehicular Networks, including Autonomous Vehicles, cybersecurity, blockchain technology and its applications.

Mohamed Hadded joins Abu Dhabi University as an assistant professor of cybersecurity engineering, in the College Engineering/CSIT Department. In 2016, he completed his PhD in computer science Engineering at Telecom SudParis College in co-accreditation with Sorbonne University (Pierre and Marie Curie Campus). Prior to joining ADU, Dr. Elhadad worked as a senior cybersecurity researcher for intelligent transportation systems at IRT SystemX Paris, France from 2021 to 2022. From 2018 to 2021, he worked as a cybersecurity research engineer at VEDECOM Institute (Versailles, France). Before that, he worked as a postdoctoral research fellow at the National Institute for Research in Digital Science and Technology (INRIA, France) from 2017 to 2018. From 2015 to 2017, he worked as a teaching and research assistant (ATER) at Paris 5 and Franche-Comté (UFC) universities.